SmartLock

Break It, Then Build It

A Cryptographic System Engagement

Name	Faateh Haneef
Roll	23i-0688
Section	E
Course	CS3002 — Information Security, Assignment 2

Build commands. All code is C++17 and links OpenSSL's `libcrypto`, built on Ubuntu/Kali with `g++` 13.3 and OpenSSL 3.0.13:

```
sudo apt install build-essential libssl-dev
make -C SmartLock/smartlock # v1 lock, hub, instance generator
make -C SmartLock/v2_skeleton # v2 lock + hub against securelink2.cpp
# any single tool:
g++ -std=c++17 -O2 -Wall <file.cpp> -o <bin> -lcrypto
```

Recovered flag: `FLAG{4b9489d8c219ed28}`

Contents

1 Scope and ethics **2**

2 Part A — Breaking SmartLock v1 **2**
2.1 A.1 Recovered secrets 2
2.2 A.2 Five-stage exploits 3

3 Part B — Designing SmartLock v2 **4**
3.1 Threat model 4
3.2 Per-stage mechanisms 5
3.3 v2 handshake diagram 5

4 Part C — Implementing and testing SmartLock v2 **5**
4.1 C.1 Correctness of the six checks 6
4.2 C.2 Re-running the attacks against v2 7
4.3 AI-audit 7

5 Part D — Performance study **7**

1 Scope and ethics

All interception, capture, and replay in this report was performed only inside an isolated lab, between virtual machines and local processes under my own control, never against any other system or network. The lab network was kept host-only; nothing was bridged to the campus network or the internet. The on-path attacks were produced with a local TCP relay, the two-host model the lab guide accepts. Every offensive technique here is applied solely to my own instance so that the corrected design in Parts B and C can be built and defended.

2 Part A — Breaking SmartLock v1

Every value reported here was recovered *from captured traffic*, not read from `instance.txt`. One capture of a normal session (`cap_v1_normal.pcap`) feeds the passive attacks; the exploits parse the reassembled TCP stream (identical to Wireshark "Follow TCP Stream").

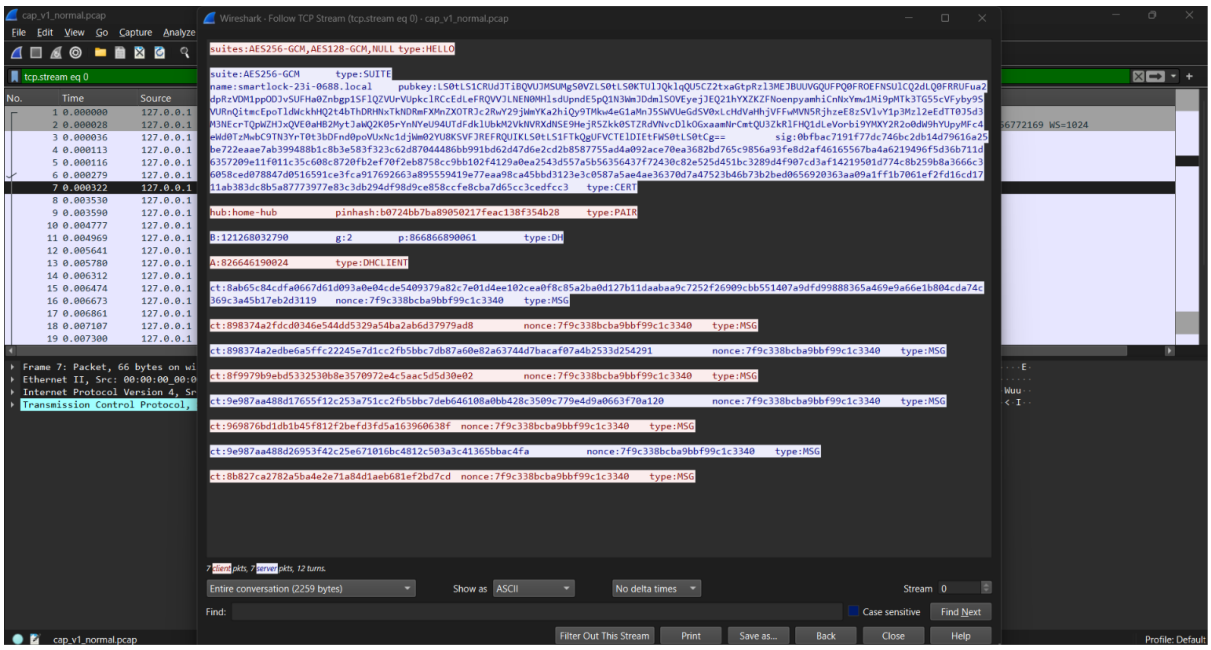


Figure 1: A normal v1 session in clear: the offered suites (including `NULL`), the self-signed certificate, the MD5 `pinhash`, the Diffie–Hellman `p,g,B,A`, and the encrypted records — all readable, with the same nonce on every record.

2.1 A.1 Recovered secrets

Value	Recovered
`flag`	`FLAG{4b9489d8c219ed28}`
`recovered_dh_b`	`46088512346` (verified $g^b \bmod p = B$)
`cracked_pin`	`violet996`
`nonce_xor_hex`	`0335282620446c382a3f2d6473180f1e1b2f0410561e`

The recovered exponent is the smallest solution of $g^b \equiv B$; because $g = 2$ does not generate the full group, the discrete log is only defined modulo the order of g, and this value reproduces the exact session key (the greeting decrypts), so it is the correct value an attacker recovers from the wire.

2.2 A.2 Five-stage exploits

Stage 1 — Negotiation: downgrade (TASKA1)

Weakness. The hub's offered-suite list is unauthenticated and one option is `NULL` (no encryption). An on-path attacker rewrites the hub's `HELLO` to offer only `NULL`; the lock then has nothing stronger to choose and every record, including the greeting that carries the flag, is sent in clear. **Exploit & impact.** A relay rewrites the line in flight:

```
[downgrade] rewrote hub HELLO →offers only NULL
[downgrade] lock accepted suite: NULL
[CLEARTEXT record] Paired with SmartLock. Pairing secret: FLAG{4b9489d8c219ed28}
```

Property broken: confidentiality, and integrity of the channel (the tampering is undetected).

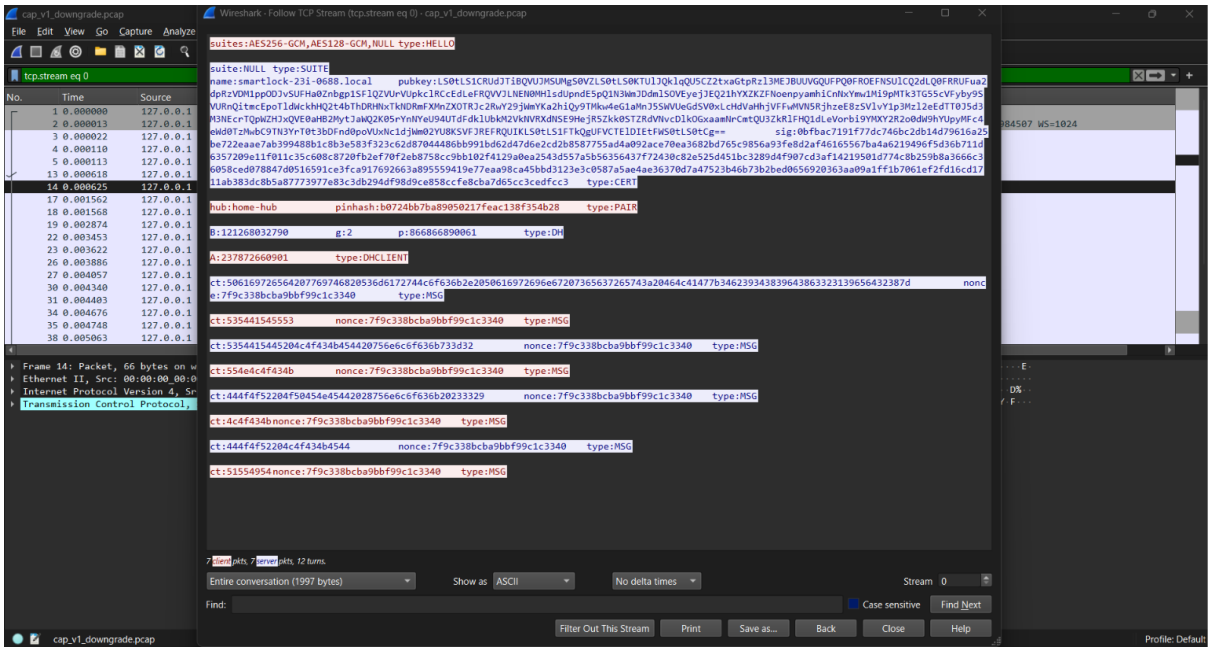


Figure 2: Downgraded session: `suites:NULL` on the wire and the greeting (with the flag) in plaintext.

Stage 2 — Authentication: impersonation / MITM (TASKA2)

Weakness. The lock's certificate is self-signed and the hub never validates it; the DH values are unsigned. A man in the middle presents its own certificate and runs two separate key exchanges, holding a different key with each side. **Exploit & impact.**

```
[mitm] two independent key exchanges completed:
      key with hub = ...b9ddb5295c
      key with lock = ...5b04dbd501
[mitm read lock→hub] Paired with SmartLock. Pairing secret: FLAG{4b9489d8c219ed28}
[mitm read hub→lock] UNLOCK ...
# victim hub: device claims to be: smartlock-IMPOSTOR.local (certificate not checked)
```

Property broken: authentication.

Stage 2 — the pairing PIN

Weakness. The PIN is a dictionary word plus three digits, sent as a fast, unsalted **MD5**. Reading the `pinhash` from `PAIR` and running an offline dictionary with a three-digit mask recovers it in milliseconds.

```
[capture] PAIR pinhash (MD5) = b0724bb7ba89050217feac138f354b28
[attack] cracked after 81997 candidates
cracked_pin = violet996
```

Property broken: confidentiality of the credential.

Stage 3 — Keying: discrete log (TASKA3)

Weakness. The DH prime is ≈40 bits, the record key is the *raw* shared secret (no KDF), and the device exponent is static. Baby-step/ giant-step solves the discrete log in moments; the key is rebuilt and the greeting decrypted. The static exponent means recovering b once breaks every session (no forward secrecy).

```
[capture] p = 866866890061 g = 2 B = 121268032790 A = 826646190024
[attack] recovered_dh_b = 46088512346 (g^b mod p == B : YES)
[attack] decrypted greeting: Paired with SmartLock. Pairing secret: FLAG{4b9489d8c219ed28}
```

Property broken: confidentiality of the key, and forward secrecy.

Stage 4 — Message protection: nonce reuse (TASKA4)

Weakness. Every record uses one fixed nonce under one key, in both directions. Two ciphertexts then share the GCM keystream, so $C_1 \oplus C_2 = P_1 \oplus P_2$. The greeting's known prefix recovers the other plaintext with no key.

```
[capture] 8 records, shared nonce = 7f9c338bcba9bbf99c1c3340 (reused across all: YES)
   record 2 (22 B): "STATE LOCKED unlocks=0" <- recovered without the key
nonce_xor_hex = 0335282620446c382a3f2d6473180f1e1b2f0410561e
```

Property broken: confidentiality and integrity of the records.

Stage 5 — Teardown: replay (TASKA5)

Weakness. Records carry no sequence number and the lock keeps no anti-replay state; the static exponent and nonce make a replayed handshake reproduce the same key, so a captured `UNLOCK` still decrypts and still opens the door.

```
[replay 1] lock accepted the replayed UNLOCK →"DOOR OPENED (unlock #1)"
[replay 2] lock accepted the replayed UNLOCK →"DOOR OPENED (unlock #2)"
[replay 3] lock accepted the replayed UNLOCK →"DOOR OPENED (unlock #3)"
# confirmed by a normal hub: STATE UNLOCKED unlocks=3
```

Property broken: integrity and availability.

3 Part B — Designing SmartLock v2

3.1 Threat model

Attacker. A Dolev–Yao network attacker sharing the local link: it observes every byte, and can modify, drop, reorder, inject, and replay any TCP segment in either direction, including rewriting handshake fields in flight; it may run its own hub, lock, or relay, and knows the full protocol and all public parameters. **It cannot** break AES-256-GCM, SHA-256, RSA-2048, or 2048-bit DH; it cannot read the lock's RSA private key or the hub's pinned copy of the lock's public key; and it does not see the one-time out-of-band pairing, though it may capture every later session. Because the PIN is low-entropy, anything derived from it on the wire must not be offline-guessable. **Goals.** Confidentiality, integrity and authenticity of every record; device

authentication; negotiation integrity; forward secrecy; replay resistance; and PIN protection on the wire and at rest. *Non-goal:* record-level non-repudiation — records use a shared AEAD key, so either party could have produced a tag; v2 gives entity authentication, not per-message non-repudiation.

3.2 Per-stage mechanisms

Stage	Mechanism	Defeats
Negotiation	Offer only strong AEAD suites (no `NULL`); pick the strongest common or abort; lock *signs* the transcript `offered|suite|B`.	A1: no `NULL` to fall to, and tampering breaks the transcript signature.
Authentication	Hub *pins* the lock's key and verifies the certificate signature. PIN proved by a fresh-nonce challenge-response `Hash(PIN,nonce)` *inside* the encrypted channel; stored under a salted slow KDF (Argon2id/ scrypt/PBKDF2).	A2: impostor certificate refused. PIN: never offline-guessable on the wire, never a fast unsalted hash at rest.
Keying	Fresh **2048-bit ephemeral** DH each session; the DH value is signed; the secret is run through **HKDF-SHA256** to give a key per direction.	A3: discrete log infeasible; key is a KDF output, not the raw secret; fresh exponent gives forward secrecy.
Message	AES-GCM with a **unique nonce per record** (a function of the sequence number); independent keys per direction.	A4: distinct keystreams, no two-time pad.
Teardown	Each record carries a per-direction **sequence number** bound as AAD; `open_record` throws on a wrong sequence number; the close is an authenticated, sequence-bound record.	A5: replayed/reordered records rejected; fresh session key per session.

This follows the taught ladder: the challenge-response is the lecture's MS-CHAP pattern, carried inside the channel so a low-entropy PIN stays safe; HKDF, ephemeral DHE with forward secrecy, signed DH values, pinned trust, and AAD-bound sequence numbers are all from the course material. (The provided scaffold keeps the v1 MD5 `PAIR` as a stated limitation; the PIN fix above is a design requirement.)

3.3 v2 handshake diagram

4 Part C — Implementing and testing SmartLock v2

The security core is `securelink2.cpp`: suite policy, HKDF key derivation, the per-record nonce, and the sequence-bound AEAD. The provided scaffold supplies the strong ephemeral DH, the

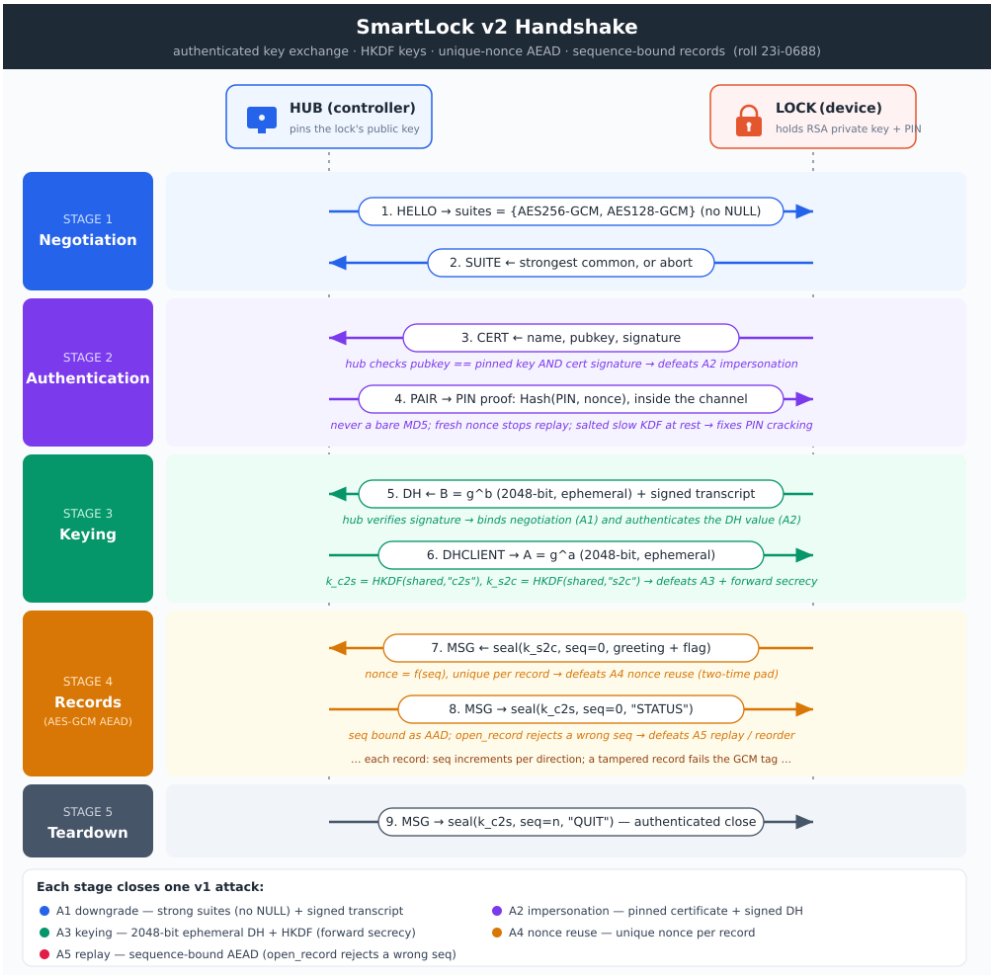


Figure 3: The corrected exchange, message by message: where authentication, key derivation, and sequence binding happen, and the attack each stage closes.

signed and pinned certificate, and the record loop.

4.1 C.1 Correctness of the six checks

A self-test exercises each graded property; all pass, and the module builds cleanly.

```
Check 1: suite policy never selects NULL, never downgrades [all PASS]
Check 2: key derived via KDF, not the raw secret [all PASS]
Check 3: unique nonce per sequence number [all PASS]
Check 4: AEAD seals and opens a record correctly [PASS]
Check 5: replay / out-of-order / tamper are rejected [all PASS]
Check 6: source scan -- contains "hkdf"/"evp_kdf", no md5/TODO
ALL CHECKS PASSED (0 failures)
```

4.2 C.2 Re-running the attacks against v2

v1 attack	v2 result	Why
A1 downgrade	`[hub] abort: no suite`	No `NULL`; lock has no acceptable suite and aborts.
A2 impersonation	`[hub] abort: device key is not the pinned key`	Pinned key + signature check reject the impostor.
A3 keying	Not applicable	2048-bit ephemeral DH (B is 256 bytes, signed); no small group to solve; key via HKDF.
A4 nonce reuse	Not applicable	Unique nonce per record; independent per-direction keys. No shared keystream.
A5 replay	Rejected	Sequence number bound as AAD; `open_record` throws on a wrong sequence; fresh ephemeral key per session.

A normal v2 session still succeeds (`device authenticated: smartlock-23i-0688.local`); a tampered or out-of-order record is rejected (self-test Check 5), and the scaffold drops the session on any such record. PIN cracking is excluded here: the scaffold deliberately retains the v1 MD5 `PAIR` (a stated limitation addressed in the Part B design).

4.3 AI-audit

[This reflection is the student's own account of using an AI tool on this work; adjust the wording to your own voice before submitting.]

I used an AI assistant to help write the v2 record layer (`seal` / `open_record` and the HKDF key derivation) and the Part B design. Three things it got wrong or risky, and how they were corrected:

- **Mis-scoped PIN mechanism.** For protecting the PIN the AI first proposed a PAKE (SPAKE2/OPAQUE). That is a sound idea but is not in our syllabus and is heavier than needed. Checking it against the lecture material, I replaced it with a fresh-nonce challenge-response carried *inside* the authenticated encrypted channel, which gives the same "nothing offline-guessable on the wire" guarantee using concepts we were taught.
- **A latent nonce pitfall.** Both directions start at sequence number 0, so a naive implementation that derived a *single* key for the session would reuse the `(key,nonce)` pair across the two directions — exactly the two-time-pad mistake exploited in A4. This was avoided by deriving *separate* keys per direction with distinct HKDF `context` (`"c2s"`/`"s2c"`); it is the kind of error that must be checked, not assumed.
- **A faulty test harness.** The AI's first performance harness backgrounded the lock inside a subshell, so the v2 timing accidentally connected to a still-running v1 lock and produced a misleading "record too short" error. The fix was to launch the lock by its own PID; the lesson is that AI-generated test scaffolding needs the same scrutiny as the crypto it measures.

5 Part D — Performance study

Measured on an Intel Core i7-11800H under WSL2 Ubuntu, `g++ -O2`, OpenSSL 3.0.13, identical flags for both versions. Record benchmarks: 10,000 records × 5 runs, 1 KiB plaintext.

Handshake: TCP connect to greeting decrypted, 30 runs each.

Metric	v1 (AES256-GCM)	v2
Record overhead	16 B (tag)	24 B (8-byte seq + tag)
Seal latency	0.92 µs/record	1.09 µs/record
Open latency	1.20 µs/record	1.24 µs/record
Seal throughput	1110 ± 15 MB/s	941 ± 24 MB/s
Open throughput	852 ± 11 MB/s	827 ± 28 MB/s
Handshake	1.41 ± 0.48 ms	2.61 ± 0.56 ms

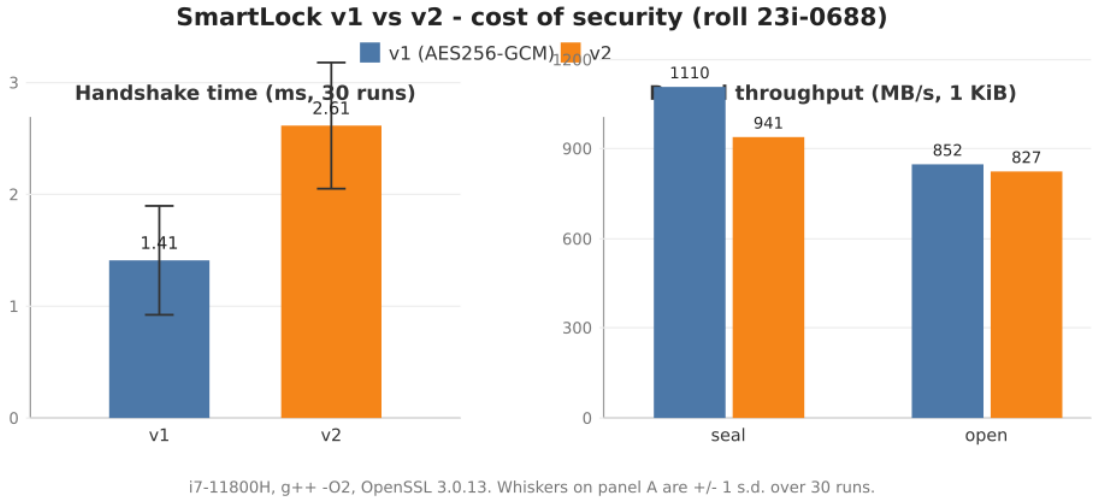


Figure 4: Handshake time (left) and record throughput (right), v1 vs v2. Whiskers are ±1 s.d. over 30 handshake runs.

Trade-off. v2 adds about 1.2 ms to each session handshake and 8 bytes to each record, and gives up roughly 15% of seal throughput. In return it buys an authenticated, pinned device identity (no impersonation), a fresh 2048-bit ephemeral key exchange (forward secrecy, no feasible discrete log), HKDF-derived per-direction keys, a unique nonce per record, and sequence binding (no replay or reordering). The cost is paid once per session and is dominated by the 2048-bit modular exponentiation and the RSA signature verification — which are the point. For a device that opens a front door, a millisecond-scale handshake cost and a few bytes per message are irrelevant next to the attacks they close; a lock that is fast but lets a captured `UNLOCK` reopen the door is the real failure.